

CREDIT CARD FRAUD DETECTION

Identifying Fraudulent Transactions with Random Forest Classification

A Machine Learning Project Report

1. Problem Statement: Victim, Attacker & Attack Surface

Credit card fraud is an adversarial problem: a legitimate cardholder's account is hijacked and used by an unauthorized third party before the cardholder or issuing bank notices. Framing the dataset around the three parties involved in every transaction clarifies exactly what the model needs to detect.

The Victim — Cardholder

The legitimate account owner, present in every transaction row

- Identified by `cc_num`, along with demographic fields: gender, job, dob, and home address (street, city, state, zip, lat, long).
- `city_pop` captures whether the cardholder lives in a dense urban area or a small town, since fraud exposure varies by location.
- The cardholder is never the perpetrator — they are the party whose account and funds are put at risk.

The Attack Surface — Merchant

The store or terminal where the stolen card is used

- Captured by `merchant`, `category` (e.g. `grocery_pos`, `shopping_net`, `entertainment`), and the merchant's GPS coordinates (`merch_lat`, `merch_long`).
- Comparing merchant location against the cardholder's home coordinates is what makes geographic anomaly detection possible.
- The merchant is a channel for the attack, not a suspect — the `'fraud_'` prefix on merchant names in the raw data is a dataset-generation artifact, not a flag.

The Attacker — Fraudster

Has no dedicated columns; only visible through behavioral traces

- No 'fraudster ID' exists in the data — their presence is inferred entirely from anomalous patterns layered on top of the victim's and merchant's data.
- Typical traces: transactions far from the cardholder's home, amounts many multiples of the cardholder's usual spend, and activity at unusual hours or high-risk merchant categories.
- The entire modeling task is to reconstruct the attacker's behavioral fingerprint from these traces well enough to flag it in real time.

The target label, `is_fraud`, marks each transaction as 0 (legitimate, executed by the real cardholder) or 1 (fraudulent, executed by the attacker). The modeling goal is to predict this label the moment a new transaction arrives, using only information available at that moment — cardholder profile, merchant profile, and transaction details.

2. Data Overview & Cleaning

- Training data: 1,296,675 transactions × 22 columns (`fraudTrain.csv`); Test data: 555,719 transactions × 23 columns (`fraudTest.csv`).
- Class balance: 1,289,169 legitimate vs. 7,506 fraudulent transactions in training — fraud makes up just 0.58% of the data, a severe class imbalance that shapes every modeling decision downstream.
- Missing values: 0 after dropping a small number of rows with null `merch_lat`; duplicate rows: 0.

Outlier check on transaction amount (`amt`): 5.19% of transactions (67,290 rows) exceed the IQR upper fence of \$193.38. These were deliberately kept rather than capped or removed — in a fraud dataset, extreme amounts are disproportionately the fraud signal itself, so trimming them would remove the exact evidence the model needs to learn from.

String cleaning: the `'fraud_'` prefix was stripped from merchant names, and hyphens/commas were normalized across merchant, category, and job text fields.

Date handling: `trans_date_trans_time` was parsed to a proper datetime and used to derive hour, `day_of_week`, and month; a stray `'#'` character in `dob` was cleaned before parsing it to a valid date.

3. Exploratory Data Analysis

Seven behavioral hypotheses were tested directly against the data to establish which signals genuinely separate fraud from legitimate activity before any modeling began:

Hypothesis	Result	Key Evidence
Fraud concentrates in a handful of high-population states	CONFIRMED	New York leads with 289 fraud cases, followed by Pennsylvania (201), Texas (187), Illinois (148).
Fraudsters reuse the same compromised merchant repeatedly	REJECTED	2,900+ fraudulent card-merchant pairs occur exactly once; repeat visits to the same merchant are almost never seen — fraudsters move on immediately.
Fraud amounts spike at night	PARTIALLY CONFIRMED	Legitimate spend stays flat (\$70–\$80) all day. Fraud amounts rise sharply from noon onward, peaking near \$780 around 8 PM — an afternoon/evening pattern, not purely nocturnal.
Gender predicts fraud risk	REJECTED	Male and female cardholders show nearly identical fraud rates (~0.6% each).
Fraud spikes on Mondays and weekends	CONFIRMED	Monday has the highest fraud count (~520+); Friday, Saturday, and Sunday all stay high (400–460+).
Online/grocery categories are high-risk	CONFIRMED	<code>shopping_net</code> (~1.8%), <code>grocery_pos</code> (~1.45%), and <code>misc_net</code> (~1.4%) have the highest fraud rates; <code>home</code> , <code>food_dining</code> , <code>health_fitness</code> are lowest.
Distance from home to merchant predicts fraud	REJECTED	Distance distributions for fraud and legitimate transactions overlap almost completely (median ~75–80 km for both).

Additional Distributional Findings

- Transaction amount is the single clearest visual signal: legitimate spending peaks sharply around 1× a cardholder's typical amount and almost never exceeds 2.5–3×, while fraudulent transactions are spread out well past 3×, 5×, and even 10× the cardholder's baseline.
- Age shows a mild bimodal fraud skew: cardholders aged roughly 45–60 and 75–85 show a slightly elevated share of fraud, while the 25–40 band is dominated by routine legitimate spending.
- Card-merchant repeatability is a strong signal on its own: fraudsters overwhelmingly hit a merchant exactly once before moving on, rarely returning to the same compromised merchant a second time.

4. Preprocessing & Feature Engineering

Beyond cleaning, five new features were constructed specifically to encode the behavioral fraud patterns surfaced in the EDA:

Engineered Feature	How It's Built	Why It Signals Fraud
age	Transaction date minus date of birth, in years	Certain age bands show disproportionate fraud exposure.
distance_km	Haversine distance between cardholder home GPS and merchant GPS	Flags geographically implausible purchases (though this dataset showed weak separation).
amt_to_avg_ratio	Transaction amount ÷ the cardholder's historical average amount	Fraud transactions are often several multiples of a cardholder's normal spend.
card_trans_count	Total transaction count per credit card number	Distinguishes lightly-used from heavily-used cards, a proxy for account activity level.
hour / day_of_week / month	Extracted from the transaction timestamp	Captures time-of-day and day-of-week fraud concentration patterns found in EDA.

Encoding & Scaling Pipeline

- Raw identifier and free-text columns (first, last, street, city, trans_num, unix_time, the raw timestamp, dob, cc_num, merchant, job, zip, lat/long, merch_lat/merch_long) were dropped after their information was folded into engineered features — keeping them would let the model memorize individual people rather than learn general fraud patterns.
- gender, category, and state were one-hot encoded, expanding the feature set to 74 columns before the target was separated out.
- An 80/20 stratified train-test split (preserving the 0.58% fraud rate in both sets) produced 1,037,340 training rows and 259,335 test rows.
- amt, distance_km, and city_pop were log-transformed (log1p) to pull in their long right tails, then all continuous features were standardized with StandardScaler fit on the training set only.

5. Model Implementation & Hyperparameter Tuning

Baseline: Logistic Regression

A Logistic Regression model with `class_weight='balanced'` (automatically upweighting the rare fraud class during training) was fit first as an interpretable baseline. On the held-out test set it reached a strong ROC-AUC of 0.9126, but its precision on the fraud class was extremely low (0.04) — a linear decision boundary struggles to separate fraud from legitimate activity given how tangled the two classes are in raw feature space, so it flags huge numbers of false alarms to catch most real fraud.

A GridSearchCV sweep over six values of the L2 regularization strength ($C = 0.001$ to 100, 5-fold CV, optimizing F1) was set up to tune this baseline further, but the search was not completed in the notebook — no best C or CV score was produced. This step is noted here as an open item rather than reported with invented numbers.

Random Forest — Why It Was Chosen

Random Forest was introduced specifically to capture the non-linear, conditional patterns the EDA surfaced — e.g. “high amount AND unusual hour AND high-risk category” is exactly the kind of interaction a tree-based ensemble captures naturally, where a linear model cannot.

Hyperparameter Choices: Baseline vs. Regularized Forest

Two Random Forest configurations were trained. The first prioritizes catching fraud aggressively; the second deliberately constrains tree growth to reduce overfitting, trading some recall for a more trustworthy, generalizable model:

Hyperparameter	Baseline RF	Regularized RF	Purpose
<code>n_estimators</code>	100	100	Number of trees averaged together; more trees stabilize predictions.
<code>max_depth</code>	10	6	Caps how many splits deep each tree can grow — lower values force simpler, less memorized trees.
<code>min_samples_split</code>	10	20	Minimum samples required in a node before it can split — higher values prevent splitting on tiny, noisy subgroups.
<code>min_samples_leaf</code>	5	10	Minimum samples required in a final leaf — higher values smooth out predictions on rare patterns.
<code>class_weight</code>	<code>balanced</code>	<code>balanced_subsample</code>	Upweights the rare fraud class during training; <code>balanced_subsample</code> recomputes weights per bootstrap sample for extra robustness.
Decision threshold	0.75	0.70	Probability cutoff above which a transaction is flagged as fraud — tuned manually rather than left at the default 0.5.

Both forests also used `class_weight` to counteract the 0.58% fraud rate, and both were evaluated at a raised decision threshold (well above the default 0.5) — because Random Forest's predicted probabilities for a rare class are typically compressed toward 0, a higher cutoff is needed to get a workable precision/recall balance.

6. Results in Detail

A critical distinction runs through this section: the first two Random Forest evaluations below were measured on the training data itself (the same rows the model learned from), which shows how well the model fits its training data but does not indicate how it performs on new, unseen transactions. Only the rows marked “Held-out test” in the summary table reflect genuine generalization performance — that distinction matters because in-sample numbers alone would overstate how well this model works in production.

Summary Comparison Across Every Stage

Model / Stage	Evaluated On	Fraud Precision	Fraud Recall	Fraud F1	ROC-AUC
Logistic Regression (baseline)	Held-out test	0.04	0.81	0.07	0.9126
Random Forest (unregularized, default 0.5 threshold)	Training data (in-sample)	0.24	0.93	0.39	0.9948
Random Forest (unregularized, threshold = 0.75)	Training data (in-sample)	0.56	0.76	0.65	—
Random Forest (unregularized, threshold = 0.75)	Held-out test	0.36	0.60	0.45	0.9617
Random Forest (regularized, threshold = 0.70)	Held-out test	0.15	0.69	0.24	0.9403

6.1 Logistic Regression — Held-out Test Set

Class	Precision	Recall	F1-Score	Support
Legitimate (0)	1.00	0.87	0.93	257,834
Fraud (1)	0.04	0.81	0.07	1,501

ROC-AUC: 0.9126. Accuracy: 87%. The model catches 81% of real fraud (good recall) but only 4% of its fraud flags are correct — in practice this would overwhelm a fraud review team with false alarms, illustrating why a linear baseline alone isn't sufficient for this problem.

6.2 Random Forest (Unregularized) — Training Data, Default Threshold

Class	Precision	Recall	F1-Score	Support
Legitimate (0)	1.00	0.98	0.99	1,031,335
Fraud (1)	0.24	0.93	0.39	6,005

ROC-AUC: 0.9948 (in-sample). These numbers reflect fit to training data and are expected to look stronger than genuine test performance — an unconstrained forest (max_depth=10 is still fairly deep relative to this feature set) has room to partially memorize training examples.

6.3 Random Forest (Unregularized) — Held-out Test Set, Threshold = 0.75

Class	Precision	Recall	F1-Score	Support
Legitimate (0)	1.00	1.00	1.00	58,108

Fraud (1)	0.36	0.60	0.45	234
------------------	------	------	------	-----

ROC-AUC: 0.9617. This is the model's genuine generalization performance: it correctly flags 60% of real fraud cases, and just over a third of its fraud flags are correct — a large step down from the in-sample numbers above, confirming some degree of overfitting in the unregularized forest.

6.4 Random Forest (Regularized) — Held-out Test Set, Threshold = 0.70

Class	Precision	Recall	F1-Score	Support
Legitimate (0)	1.00	0.98	0.99	58,108
Fraud (1)	0.15	0.69	0.24	234

ROC-AUC: 0.9403. Constraining the trees (shallower depth, larger minimum leaf/split sizes) raised recall from 60% to 69% — catching more real fraud — but precision dropped from 0.36 to 0.15, meaning more false alarms per true fraud caught. This is a genuine precision/recall trade-off, not a strictly better model: which configuration is preferable depends on whether the business cost of a missed fraud case outweighs the cost of an extra manual review.

Note on evaluation data: this held-out evaluation and the one in Section 6.3 were run against a variable (`test_df`) built inside the notebook but not shown being loaded from `fraudTest.csv`, and its row count (58,342) is smaller than the original test file (555,719 rows). The relative comparison between the two Random Forest configurations is still valid since both were scored on the same rows, but the exact provenance of this particular subset should be confirmed before quoting these numbers as the model's official test-set performance.

7. How the Model Predicts a New Transaction

In production, this pipeline would score a new transaction the moment it's submitted for authorization, before the charge is finalized. The end-to-end path from a raw transaction to a fraud decision looks like this:

1. Capture the same raw fields used in training: cardholder details (dob, gender, home address/coordinates, city_pop), merchant details (name, category, coordinates), and the transaction itself (amount, exact timestamp).
2. Recompute the engineered features live: age from dob and the current timestamp; distance_km via the Haversine formula between cardholder and merchant coordinates; amt_to_avg_ratio using that card's running historical average; card_trans_count from that card's running transaction count; and hour / day_of_week / month from the current timestamp.
3. One-hot encode gender, category, and state using the exact same columns the model was trained on, filling any category unseen during training with 0.
4. Log-transform amt, distance_km, and city_pop, then apply the StandardScaler fitted during training (never refit on new data) to put every feature back on the scale the model expects.
5. Feed the finished feature vector into the trained Random Forest, which outputs a fraud probability between 0 and 1 rather than a hard yes/no.
6. Compare that probability to the chosen decision threshold (0.70–0.75 in this project, rather than the default 0.5) to produce the final label: flagged for review, or passed through as legitimate.

What the Output Actually Means

A probability, not a verdict: the model never outputs “fraud” or “not fraud” directly — it outputs a likelihood. The threshold chosen in Section 5 is what converts that likelihood into an action, and that threshold can be moved up or down after deployment without retraining the model, to trade precision against recall as business needs change.

A flagged transaction is a request for review, not proof of guilt: given the fraud-class precision of 0.15–0.36 in the held-out evaluations, a large share of flagged transactions will turn out to be legitimate. This model is best used to prioritize a human fraud-review queue or trigger a secondary verification step (e.g. an SMS confirmation), not to auto-decline purchases outright.

Coverage depends on the tail behavior it learned: because the model's strongest signals are amount-relative-to-baseline, merchant category, and time-of-day/week, a fraudster who keeps charges small, sticks to low-risk categories, and avoids the flagged time windows is inherently harder for this model to catch — a limitation worth stating plainly rather than implying the model catches all fraud patterns equally well.

New cardholders and cards are a blind spot: `amt_to_avg_ratio` and `card_trans_count` both depend on transaction history building up over time. A brand-new card has no baseline yet, so these two features — among the most useful in the model — are far less informative for a customer's very first few transactions.

Prepared as part of an academic machine learning project.